Project Proposal

Project: AI-Powered Predictive Maintenance for Industrial Equipment

Objective: Develop a machine-learning system that uses engine sensor data to provide early warnings of equipment failure. The core prediction will classify whether an engine is likely to fail within the next 30 operational cycles; a secondary regression model will estimate Remaining Useful Life (RUL) if it can be completed without delaying the core system.

Description: We will use the NASA C-MAPSS FD001 dataset. The work includes data analysis, preprocessing, time-series feature engineering, and comparison of Random Forest, XGBoost, and LSTM. The selected model will be connected to a FastAPI service and Streamlit dashboard. A live-style demo will replay test-engine sensor readings cycle by cycle, and MLflow will track experiments and model versions.

Tools & Design:
Main workflow:
Sensor Data → Preprocessing → Feature Engineering → Model Training → Prediction API → Dashboard
The data will be split by engine to avoid data leakage.
Classification will use Precision, Recall, F1-score, and ROC-AUC. RUL will use MAE/RMSE.
The first deployment target is a working local/Docker application.

Technologies to Use: Python, Jupyter Notebook, Pandas, NumPy, Matplotlib/Plotly, scikit-learn, XGBoost, TensorFlow/Keras, FastAPI, Streamlit, MLflow, Git/GitHub, and Docker.

Team Members & Responsibilities:

1- **Malak Mohamed Elsaid - Project Coordination & Data Engineering:** planning, GitHub workflow, dataset setup, architecture, and integration.

2- **Sandra Sameh Saad - Data Analysis & Feature Engineering:** EDA, data cleaning, sensor visualizations, and time-series features.

3- **Razan Ahmed Ramadan - Machine Learning:** target preparation, Random Forest/XGBoost, tuning, and model comparison.

4- **Dareen Mohamed Elsayed - Time-Series & Evaluation:** LSTM, sequence preparation, validation, metrics, and error analysis.

5- **Jana Ahmed Farouk - Deployment & MLOps:** FastAPI, Streamlit, MLflow, Docker, logging, and basic monitoring.

All members will participate in testing, documentation, presentation, and final integration.

Week 1-3: Gathering Requirements & Data Collection

- **Malak:** finalize the team, project scope, requirements, GitHub repository, task plan, and initial architecture.
- **Malak & Sandra:** load the FD001 dataset and finalize the data-collection plan.
- **Sandra:** perform EDA, data-quality checks, and sensor/degradation visualizations.
- **Sandra & Razan:** clean the data, handle missing/erroneous values, and define the 30-cycle failure label and RUL target.
- **Dareen:** define the engine-based train/validation/test split (to avoid data leakage) and the LSTM sequence format.
- **Jana:** set up the MLflow tracking server and the initial experiment-logging structure.

Deliverables:

- Draft scope, requirements, and architecture document
- Dataset Exploration Report
- EDA Notebook
- Preprocessed FD001 data with defined labels and data split

Week 4-7: Analysis & Design

- **Sandra:** engineer rolling averages, sensor rate-of-change, and degradation trends, and apply PCA to identify key failure indicators.
- **Razan:** design the classification pipeline (Random Forest & XGBoost) and the model-comparison strategy.
- **Dareen:** design the LSTM sequence-preparation pipeline and its evaluation approach.
- **Jana:** design the system architecture: FastAPI service, Streamlit dashboard layout, and the MLOps pipeline (MLflow, Docker).
- **Malak:** consolidate the individual designs into an end-to-end architecture diagram and confirm module interfaces.

Deliverables:

- Advanced Analysis Report (key failure indicators, PCA results)
- Feature Engineering Summary
- Degradation-pattern and sensor-anomaly visualizations
- End-to-end system architecture and design document

Week 8-11: Implementation

- **Razan:** implement and tune Random Forest and XGBoost, using Grid Search/Random Search for hyperparameters.
- **Dareen:** implement and train the LSTM model using time-series cross-validation.
- **Razan & Dareen:** compare models using Precision, Recall, F1-score, ROC-AUC (and MAE/RMSE for RUL) and explore ensemble methods.
- **Jana:** build the FastAPI prediction service and Streamlit dashboard, connect the selected model, and containerize the application with Docker.
- **Jana:** implement the simulated live sensor-replay demo and prediction logging.
- **Malak:** integrate the data, model, and serving code into a single reusable pipeline and maintain the GitHub workflow.

Deliverables:

- Model Evaluation Report comparing Random Forest, XGBoost, and LSTM
- Final selected model with its saved preprocessing pipeline
- Deployed FastAPI + Streamlit application (Dockerized)
- MLflow experiment tracking and model registry
- Simulated live sensor-stream demo

Week 12-14: Testing

- **Malak & Dareen:** run end-to-end integration tests (API, dashboard, model pipeline), log issues, and apply fixes.
- **Jana:** test the containerized deployment and add basic drift/performance-monitoring checks with alerting.
- **Sandra & Razan:** validate the final EDA, feature-engineering, and model-result sections against the test outcomes.
- **All members:** conduct a full walkthrough of the dashboard and live-replay demo.

Deliverables:

- Test report (unit, integration, and UI testing)
- Monitoring and drift-check setup
- Verified, stable end-to-end application

Week 15: Release & Final Presentation

- **Malak:** assemble the final report, README, technical documentation, and deployment/retraining guide.
- **All members:** prepare slides, rehearse the live demo, and finalize individual contribution summaries.
- **Team:** deliver the final presentation and live demonstration to the instructor and stakeholders.

Deliverables:

- Final Project Report
- README and technical documentation
- Final presentation with live demonstration
- Complete, organized GitHub repository
